HANDS-ON ASSIGNMENT

AI Customer Support Automation

Build a multi-agent support desk with AutoGen, Python and Streamlit

FRAMEWORK	LANGUAGE	UI	AGENTS	LEVEL
AutoGen	**Python 3.10+**	**Streamlit**	**4 agents**	**Intermediate**

1 The Scenario

A growing online store receives hundreds of customer complaints every day: late deliveries, refund requests, damaged products, billing problems and login issues. The support team answers the same questions again and again, and the difficult cases get lost in the queue.

Your job is to build an **AI Customer Support Desk** where a team of AI agents works together. They read the complaint, find the answer in the company's own documents, write a helpful reply, and hand the hard cases to a human, keeping a record of every ticket so the team can learn from it later.

What you will learn
Designing agents with clear, single responsibilities · Passing work between agents in AutoGen · Grounding answers in company documents (simple RAG) · Escalation rules · Logging to a file · Wrapping it all in a Streamlit app

2 Meet the Four Agents

Agent	Responsibility	Input	Output
Customer Query Agent	Understands the customer's request: what the problem is, how urgent it is, and how the customer feels.	Raw complaint text	Category, urgency, sentiment, short summary
Knowledge Base Agent	Searches the company documents (FAQs, policies) and pulls out only the relevant passages.	Summary + category	Matching passages with the source file name
Resolution Agent	Writes a clear, polite reply that solves the problem, using only the facts the Knowledge Agent found.	Summary + passages	Customer-ready response + confidence
Log Agent	Records every ticket to a file. If the case is complicated, marks it **Escalated** and hands it to human support.	Everything above	A log row + escalation decision

3 How a Complaint Flows

1. **Customer submits a complaint**
   Typed into the Streamlit app.
2. **Query Agent identifies the issue**
   Category, urgency and sentiment are extracted.
3. **Knowledge Agent searches the docs**
   Relevant policy or FAQ passages are retrieved.
4. **Resolution Agent writes the reply**
   Grounded only in the retrieved passages.
5. **Log Agent records the ticket**
   Saved to TXT / CSV / Excel; complex cases escalated to a human.

4 Tech Stack

Layer	Tool	Notes
Agents	AutoGen (`autogen-agentchat`, v0.4+ API)	Use `AssistantAgent` for each agent and a team such as `RoundRobinGroupChat` to run them in order.
LLM	OpenAI / Azure OpenAI / Gemini / Groq / Ollama	Any model client AutoGen supports. Keep the API key in a `.env` file.
Knowledge base	Plain .txt / .md files (+ optional vector store)	Start with keyword search; upgrade to embeddings + ChromaDB/FAISS for bonus marks.
Logging	Python `csv` / `pandas` / `openpyxl`	One row per ticket. TXT, CSV or Excel are all accepted.
UI	Streamlit	Complaint form, live agent steps, final reply, escalation badge, log viewer.

5 Your Tasks, Step by Step

Task 1 · Set up the project

- Create a new GitHub repository and a Python virtual environment.
- Install `autogen-agentchat`, `autogen-ext[openai]` (or your provider's extension), `streamlit`, `python-dotenv` and `pandas`.
- Add a `.env` file for your API key and a `.gitignore` that excludes it.

Task 2 · Create the company knowledge base

- Inside a `knowledge_base/` folder, write at least **4 short documents** for a made-up company, for example: *shipping_policy*, *refund_policy*, *account_and_login*, *product_faq*.
- Include real-looking details (refund window in days, delivery timelines, steps to reset a password) so answers can be checked.

Task 3 · Build the four agents in AutoGen

- Give every agent a clear **name**, a one-line **description** and a focused **system message**.
- **Query Agent**: return the category, urgency (Low / Medium / High), sentiment and a one-line summary.
- **Knowledge Agent**: call a Python tool such as `search_kb(query)` that reads your documents and returns matching passages with the file name.
- **Resolution Agent**: write the reply using only retrieved facts. If nothing relevant was found, say so honestly instead of guessing.
- **Log Agent**: call a tool such as `log_ticket(...)` that appends a row to your log file, and decide whether to escalate.

Task 4 · Connect the agents into a team

- Run the agents in the order Query → Knowledge → Resolution → Log.
- Add a termination condition so the conversation stops after the Log Agent finishes (for example a `TextMentionTermination` on a word like *LOGGED*, plus a max-message limit as a safety net).

Task 5 · Define the escalation rule

- Escalate to human support when **any** of these is true: urgency is High, the customer is very angry, the Knowledge Agent found no matching document, the issue involves legal threats, fraud or payment disputes, or the Resolution Agent's confidence is low.
- Escalated tickets must show status **Escalated** in the log and a clear badge in the UI.

Task 6 · Build the Streamlit UI

- A text area to enter the complaint (plus optional customer name / order ID) and a **Submit** button.
- Show each agent's output as it happens, in expandable sections or a step-by-step timeline.
- Show the final customer reply in a highlighted box, with a green **Resolved** or red **Escalated** badge.
- A **Ticket Log** section that displays the log file as a table and lets you download it.

6 Suggested Project Structure

```
customer-support-agents/
├── app.py                  # Streamlit UI
├── agents/
│   ├── query_agent.py      # Customer Query Agent
│   ├── knowledge_agent.py  # Knowledge Base Agent + search_kb tool
│   ├── resolution_agent.py # Resolution Agent
│   └── log_agent.py        # Log Agent + log_ticket tool
├── team.py                 # wires the agents into one AutoGen team
├── knowledge_base/
│   ├── shipping_policy.md
│   ├── refund_policy.md
│   ├── account_and_login.md
│   └── product_faq.md
├── logs/
│   └── tickets.csv         # created automatically
├── screenshots/            # UI screenshots for the README
├── .env.example            # OPENAI_API_KEY=your-key-here
├── .gitignore
├── requirements.txt
└── README.md
```

You may organise it differently, but keep each agent easy to find and keep secrets out of the code.

7 Log File Format

Every submitted complaint becomes one row. Use at least these columns:

Column	Example
ticket_id	TKT-0007
timestamp	2026-11-02 14:35:10
customer_name / order_id	Priya · ORD-4512
complaint	My order is 10 days late and nobody replies to my emails.
category · urgency · sentiment	Shipping · High · Angry
kb_sources	shipping_policy.md
response	We're sorry for the delay. Your order …
status	Escalated
escalation_reason	High urgency + angry customer

8 Test Your System With These Complaints

#	Complaint	Expected result
1	"Where is my order? It was supposed to arrive yesterday."	Shipping · answered from shipping policy · **Resolved**
2	"I want to return the shoes I bought last week. How do I get a refund?"	Refund · steps + refund window from policy · **Resolved**
3	"I forgot my password and can't log in."	Account · reset steps · **Resolved**
4	"The blender I received is broken. Can I get a replacement?"	Product / Refund · replacement process · **Resolved**
5	"You charged my card three times! Fix this now or I'm contacting my bank and a lawyer."	Billing · High · Angry · **Escalated**
6	"Do you sell gift cards for a company of 500 employees with custom branding?"	No matching document · honest reply · **Escalated**

Add at least **two test cases of your own** and include their screenshots in your README.

9 Before You Submit: Checklist

- ☐ All four agents are built with AutoGen and each has a clear, single job.
- ☐ The Knowledge Agent answers from *your* documents and shows which file it used.
- ☐ The Resolution Agent does not invent policies that are not in the knowledge base.
- ☐ Every ticket is saved to a TXT, CSV or Excel file with the columns from Section 7.
- ☐ Complicated cases are marked **Escalated** with a reason.
- ☐ The Streamlit app runs with one command: `streamlit run app.py`.
- ☐ No API keys are committed to GitHub (`.env` is in `.gitignore`).
- ☐ README explains setup, how to run, the architecture and includes screenshots.

10 Bonus Challenges (Optional)

- Replace keyword search with embeddings and a vector store (ChromaDB or FAISS).
- Use `SelectorGroupChat` so the model decides which agent speaks next.
- Add a **human-in-the-loop** step with AutoGen's `UserProxyAgent` so a support person can approve escalated replies.
- Add a small dashboard in Streamlit: tickets per category, resolved vs escalated, average urgency.
- Detect the customer's language and reply in the same language.
- Deploy the app (Streamlit Community Cloud or your own VPS with Docker).

11 How to Submit

01 GitHub
Push the complete code to a **public** repository with a clear README, `requirements.txt`, `.env.example` and a `screenshots/` folder. Copy the repository link.

02 WhatsApp
Post screenshots of your working app in the batch WhatsApp group: one **Resolved** ticket, one **Escalated** ticket and the **Ticket Log** table. Add your GitHub link.

03 Skool Community
Create a post in the Skool community with the same screenshots, your GitHub link and 3–4 lines on what you built and the hardest thing you solved.

Important
Never share your API key in screenshots, posts or code. If a key was ever pushed to GitHub, revoke it from your provider's dashboard and create a new one.

12 Evaluation

Area	What we look for	Marks
Agent design	Four AutoGen agents with clear roles and good system messages	20
Knowledge grounding	Answers come from the documents; sources shown; no made-up policies	20
Workflow & escalation	Correct order, clean termination, sensible escalation rule	20
Logging	Complete, readable log file with all required columns	15
Streamlit UI	Easy to use, shows each agent step, badges and log viewer	15
Code & README	Clean structure, no secrets, setup steps and screenshots	10
Total		**100**

Build it, break it, improve it.
Start simple: get one complaint flowing through all four agents end to end, then add the UI, then the polish. Share your progress in the community; questions are welcome at every step.